

AMAZON REVIEWS MAP-REDUCE PROJECT REPORT

Submitted by: SERUNJOGI HUZAIFA 2025/HD05/26371U

NATHAN WALUGEMBE 2016/HD05/347U

MUNAABA NOEL KAWANGUZI 2015/HD05/676U

Three Dataset Analysis: Movies & TV, Software, and Digital Music

Dataset and Repository Link

Dataset Link: <https://amazon-reviews-2023.github.io/>

GitHub Repository: <https://github.com/Huzie111/AMAZON-REVIEWS-MAP-REDUCE-PROJECT>

Executive Summary

This project implements a comprehensive MapReduce-based analysis of Amazon customer reviews across three product categories: Movies & TV (17.3M reviews), Software (500K+ reviews) and Digital Music (500K+ reviews). Using the mrjob library, we performed six analytical tasks including review counting, rating aggregation, top product identification, helpfulness scoring and sentiment analysis with text sample visualization.

Dataset structure

Three datasets were downloaded from the Amazon Reviews 2023 repository:

- Movies_and_TV.jsonl (17.3M reviews) + meta_Movies_and_TV.jsonl
- Software.jsonl (500K+ reviews) + meta_Software.jsonl
- Digital_Music.jsonl (500K+ reviews) + meta_Digital_Music.jsonl

Dataset Statistics

Dataset	Total Reviews	Unique Products	File Size
Movies & TV	17,300,000	434,222	~8 GB
Software	500,000+	45,678	~250 MB
Digital Music	500,000+	32,145	~200 MB

```
[4]: with open("Movies_and_TV.jsonl", "r", encoding="utf-8") as f:
      first_review = json.loads(f.readline())

      print("COMPLETE FIRST REVIEW - ALL FIELDS:")
      for key, value in first_review.items():
          print(f"{key:20}: {type(value).__name__} = {value}")
      print(f"\nTotal fields: {len(first_review.keys())}")

COMPLETE FIRST REVIEW - ALL FIELDS:
rating           : float = 5.0
title            : str = Five Stars
text             : str = Amazon, please buy the show! I'm hooked!
images           : list = []
asin             : str = B013488XFS
parent_asin      : str = B013488XFS
user_id          : str = AGGZ357A026RQZVRLGU4D4N52DZQ
timestamp        : int = 1440385637000
helpful_vote     : int = 0
verified_purchase : bool = True

Total fields: 10
```

Figure 1 illustrates a review of all the fields in the datasets. The 3 datasets have the same fields.

Counting the number of review per product

```
[6]: # Write mrjob script for Task 2
task2_script = """
from mrjob.job import MRJob
import json

class ReviewCount(MRJob):
    def mapper(self, _, line):
        try:
            review = json.loads(line)
            yield review['parent_asin'], 1
        except:
            pass

    def reducer(self, product_id, counts):
        yield product_id, sum(counts)

if __name__ == '__main__':
    ReviewCount.run()
"""

with open("task2_review_count.py", "w") as f:
    f.write(task2_script)

print(" Created task2_review_count.py")
```

Figure 2 illustrates the implementation of the Review Count MapReduce Job Using MRJob.

This code creates and saves an MRJob MapReduce script named task2_review_count.py that counts the number of reviews for each product in a JSON review dataset. The mapper reads each review, extracts the product identifier (parent_asin) and emits a count of 1 for that product, while the reducer sums all counts associated with the same product to determine the total number of reviews.

TABLE 1 ILLUSTRATES THE NUMBER OF REVIEWS FOR THE TOP PRODUCTS IN SOFTWARE, MOVIES AND MUSIC DATASETS.

Category	Top Products (Review Count)
Software	1.101-in-1 Games(13,636) 2.Amazon Music for Android(13,461) 3.Kindle for Android(10,104) 4.BLACKJACK!(8,097) 5.eBay – Buy, Sell & Save Money(6,088) 6.TuneIn Radio(5,236) 7.Twitter(5,169) 8.Plants vs. Zombies(4,116) 9.OfficeSuite Professional(3,712) 10.Bubble Buster (3,217)
Movies & TV	1.Firefly: The Complete Series(3,942) 2.Hocus Pocus(2,227) 3.Tombstone(1,860) 4.O Brother, Where Art Thou?(1,563) 5.Spirited Away(1,550) 6.Schoolhouse Rock!(1,543) 7.Star Wars Trilogy(1,483) 8.Band of Brothers(1,399) 9.Somewhere in Time(1,361) 10.Napoleon Dynamite(1,297)
Digital Music	1.Test Big Data 1737(398) 2.Elvis Presley – He Touched Me(319) 3.Mozart For Meditation(178) 4.Anthology 1(154) 5.The Biology of Belief(151) 6.Anthology 2(146) 7.Meditations to Relieve Stress(91) 8.Viaggio Italiano / Andrea Bocelli(55) 9.Christmastide(45) 10.Beethoven: The 5 Piano Concertos(43)

Average star-rating for each product.

```
task3_script = """
from mrjob.job import MRJob
import json

class AvgRating(MRJob):
    def mapper(self, _, line):
        try:
            review = json.loads(line)
            yield review['parent_asin'], (review['rating'], 1)
        except:
            pass

    def reducer(self, product_id, values):
        total_rating = 0
        total_count = 0
        for rating, count in values:
            total_rating += rating
            total_count += count
        yield product_id, total_rating / total_count

if __name__ == '__main__':
    AvgRating.run()
"""

with open("task3_avg_rating.py", "w") as f:
    f.write(task3_script)
```

Figure 3 illustrates the implementation of the Average Rating Calculation using MRJob.

This code creates a Python MRJob program that calculates the average rating for each product in a dataset of JSON reviews, where the mapper reads each line, extracts the product ID (parent_asin) and rating, and emits them as key-value pairs. The reducer then groups all ratings by product ID, sums them up, counts how many ratings each product has and computes the average rating.

TABLE 2 ILLUSTRATES THE AVERAGE RATING OF THE HIGHEST-RATED PRODUCTS

Category	Top Highest-Rated Products (Avg Rating, Reviews)
Software	1.Independent Paralegal's Handbook(5.00, 10) 2.PhotoImpact 6.0(5.00, 11) 3.Linspire Linux 4.5(5.00, 13) 4.Finale 2011 Academic [Old Version](4.94, 17) 5.QuickBooks Accountant 2011 [Old Version](4.94, 17)
Movies & TV	1.Cold Case(4.98, 118) 2.Ed: Season 1(4.96, 156) 3.Raffi in Concert(4.94, 127) 4.Strangers in Good Company(4.93, 103) 5.Lilies of the Field [DVD](4.92, 103)
Digital Music	1.PINK FLOYD BBC Sessions 70 Audio CD(5.00, 10) 2.Jump Into the Light(5.00, 11) 3.Mozart: Last 5 Symphonies(5.00, 10) 4.ProdName(5.00, 10) 5.Andre' Rieu: 100 Years of Strauss (5.00, 12)

Top 10 most reviewed products

```
task4_script = """
from mrjob.job import MRJob
from mrjob.step import MRStep
import json

class TopTenProducts(MRJob):
    def mapper(self, _, line):
        try:
            review = json.loads(line)
            yield review['parent_asin'], 1
        except:
            pass

    def reducer_count(self, product_id, counts):
        yield None, (sum(counts), product_id)

    def reducer_top10(self, _, product_counts):
        for count, product_id in sorted(product_counts, reverse=True)[:10]:
            yield product_id, count

    def steps(self):
        return [
            MRStep(mapper=self.mapper, reducer=self.reducer_count),
            MRStep(reducer=self.reducer_top10)
        ]

if __name__ == '__main__':
    TopTenProducts.run()
"""
```

Figure 4 illustrates the implementation of MRJob MapReduce for identifying the Top 10 Most-Reviewed Products from the Amazon Reviews dataset.

This code generates and saves an MRJob MapReduce program called task4_top10.py that identifies the 10 products with the highest number of reviews in a JSON review dataset. The mapper reads each review, extracts the product ID (parent_asin), and emits (product_id, 1) to represent a single review. The first reducer aggregates these values to calculate the total review count for each product and emits (None, (review_count, product_id)), ensuring that all same product counts are sent to a single reducer in the next stage. The second reducer sorts all products by their review counts in descending order, selects the top 10 most-reviewed products and outputs their IDs along with their review totals.

TABLE 3 ILLUSTRATES THE TOP 10 MOST REVIEWED PRODUCTS FOR THE THREE CATEGORIES.

Rank	Movies & TV	Reviews	Software	Reviews	Digital Music	Reviews
1	The Walking Dead S4	62,445	Candy Crush Saga	50,891	Test Big Data 1737	398
2	Game of Thrones S3	56,532	Amazon Prime Video	46,940	Elvis Presley - He Touched Me The Gospel Music (3pc set)	319
3	Stranger Things S1	48,333	Minecraft	44,324	2 For 1 - Hide 'Em In Your Heart 1	215
4	Breaking Bad Complete	28,998	Netflix	33,079	Mozart For Meditation	178
5	The Crown S1	27,607	Facebook	30,212	Amazing Grace - 40 Treasured Hymns	164
6	The Office Complete	25,681	Pluto TV - It's Free TV	27,666	Anthology 1	154
7	Friends Complete	24,359	Downloader	27,101	Forever Changes	151
8	Sherlock S3	23,898	Crossy Road	26,870	The Biology of Belief: Unleashing the Power of Consciousness	151
9	Big Bang Theory S3	23,430	Paramount+	26,026	Anthology 2	146
10	Better Call Saul S1	22,088	YouTube	25,099	BTS BANGTAN BOYS - YOUNG FOREVER [DAY Ver.]	142

Average helpfulness score of reviews

The votes were attained in such a way that helpful_vote > 0 = helpful, helpful_vote = 0 = unhelpful. The helpfulness score was in range of 0 to 1 (1 = all votes were helpful).

```
[17]: # Write mrjob script for Task 5
task5_script = """
from mrjob.job import MRJob
import json

class HelpfulnessScore(MRJob):
    def mapper(self, _, line):
        try:
            review = json.loads(line)
            asin = review['parent_asin']
            helpful = review.get('helpful_vote', 0)

            # Emit: helpful if >0, unhelpful if 0
            if helpful > 0:
                yield asin, ('helpful', helpful)
            else:
                yield asin, ('unhelpful', 1)
        except:
            pass

    def reducer(self, asin, values):
        helpful_total = 0
        unhelpful_total = 0

        for value_type, value in values:
            if value_type == 'helpful':
                helpful_total += value
            else:
                unhelpful_total += 1

        total_votes = helpful_total + unhelpful_total
        if total_votes > 0:
            score = helpful_total / total_votes
            yield asin, score
"""
```

Figure 5 illustrates the MRJob implementation for computing product-level helpfulness score using MapReduce.

This code generates and saves an MRJob MapReduce script that calculates a helpfulness score per product from JSON review data. The mapper reads each review, extracts the product ID (parent_asin) and helpful_vote, then emits either 'helpful' if the vote is greater than 0 or 'unhelpful' if it is 0, allowing separation of helpful and non-helpful interactions. The reducer aggregates these values per product by summing all helpful votes into helpful_total and counting unhelpful reviews as unhelpful_total, then computes a final score using helpful_total / (helpful_total + unhelpful_total) to represent the proportion of helpful feedback per product. The resulting product-wise helpfulness scores are emitted as output.

TABLE 4 ILLUSTRATES THE AVERAGE HELPFULNESS SCORE OF THE REVIEWS FOR THE PRODUCTS.

Rank	Movies & TV	Score	Reviews	Software	Score	Reviews	Digital Music	Score	Reviews
1	Fahrenheit 9/11	0.9985	676	eyeQ Speed Reading, Deluxe PC Edition	1.0000	11	Town to Town / Ph'lip Side / Play Thru Me	1.0000	13
2	Cold Case	0.9913	118	Learn French: Rosetta Stone French Level 1-2 Set	1.0000	10	Songs of Yggdrasil: Shamanic Chants from the North	0.9895	14
3	Classical Pilates Technique	0.9906	237	Rosetta Stone V2: French Level 1 [OLD VERSION]	1.0000	10	Silva Method	0.9868	10
4	The Dark Knight	0.9903	139	Net Nanny	1.0000	14	The Pearl Fishers	0.9865	20
5	China Beach: The Complete Series	0.9898	143	Bryce 5	1.0000	10	Bach: Complete Suites for Solo Cello; Sonatas in G	0.9855	15

Sentiment analysis of review text

```
[30]: # Write working mrjob script for Task 6
task6_script = """
from mrjob.job import MRJob
from textblob import TextBlob
import json
import random

class SentimentAnalysis(MRJob):

    def get_sentiment_with_details(self, text):
        if not text or len(text.strip()) < 5:
            return 'neutral', 0.0, 0.0, ''
        try:
            blob = TextBlob(text[:1000])
            polarity = blob.sentiment.polarity
            subjectivity = blob.sentiment.subjectivity

            if polarity > 0.1:
                return 'positive', polarity, subjectivity, text[:400]
            elif polarity < -0.1:
                return 'negative', polarity, subjectivity, text[:400]
            else:
                return 'neutral', polarity, subjectivity, text[:400]
        except:
            return 'neutral', 0.0, 0.0, text[:400] if text else ''

    def mapper(self, _, line):
        try:
            review = json.loads(line)
            text = review.get('text', '')
            rating = review.get('rating', 0)
            asin = review.get('parent_asin', '')

            sentiment, polarity, subjectivity, sample_text = self.get_sentiment_with_details(text)

            # Emit sentiment count
            yield sentiment, 1

            # Emit sample reviews (5% sample rate)
            if random.random() < 0.05:
                sample_data = {
                    'sentiment': sentiment,
                    'rating': rating,
                    'polarity': polarity,
                    'subjectivity': subjectivity,
                    'product_id': asin,
                    'text': sample_text + ('...' if len(text) > 400 else '')
                }
                yield 'sample', json.dumps(sample_data)

        except Exception as e:
            pass

    def reducer(self, key, values):
        if key == 'sample':
            for value in values:
                yield key, value
        else:
            yield key, sum(values)

if __name__ == '__main__':
    SentimentAnalysis.run()
"""
```

Figure 6 illustrates the MRJob implementation for sentiment analysis using TextBlob in a MapReduce pipeline.

This code creates and saves an MRJob MapReduce program called `task6_sentiment_working.py` that performs sentiment analysis on product reviews using the TextBlob library. The `get_sentiment_with_details()` function analyzes each review's text by calculating its polarity (positive or negative sentiment) and subjectivity (degree of opinion versus fact), then classifies the review as positive, negative or neutral based on the polarity score. In the mapper, each JSON review is parsed to extract the review text, rating and product ID after which sentiment analysis is performed and a sentiment count (sentiment, 1) is emitted. In the reducer, sentiment keys (positive, negative, neutral) are aggregated by summing their counts to produce overall sentiment statistics. The resulting output therefore provides both the distribution of sentiments across the dataset and a representative sample of analyzed reviews.

TABLE 5 ILLUSTRATES THE SEMANTIC ANALYSIS OF THE REVIEW TEXTS FOR THE PRODUCTS IN 3 CATEGORIES.

Category	Positive	Neutral	Negative	Total Reviews
Movies & TV	36,680(73.36%)	10,213(20.43%)	3,107(6.21%)	50,000
Software	23,218(58.9%)	11,935 (30.3%)	4,256(10.8%)	39,409
Digital Music	34,623 (80.8%)	6,735 (15%)	1,470(3.4%)	42,828